

Programmation Fonctionnelle

Cours 10

Introduction au Lambda Calcul

Delia Kesner

Alonzo Church (1903 - 1995)



Motivation

- ▶ Le λ -calcul est un modèle mathématique minimal de la **computation**.
- ▶ Introduit par **Alonzo Church** dans les années 1930.
- ▶ C'est la base théorique des **langages fonctionnels modernes** (OCaml, Haskell, Lisp...).
- ▶ Permet d'étudier les notions de *fonction*, *application* et *réduction*.

Syntaxe du λ -calcul

La grammaire des termes:

$$\begin{array}{ll} t, u ::= & x \quad \text{(variable)} \\ & t \ u \quad \text{(application)} \\ & \lambda x. t \quad \text{(abstraction)} \end{array} \quad \left| \right.$$

Example : $\lambda x. x$ (identité) $(\lambda x. x \ x)$ (duplicateur).

En OCaml:

- ▶ Application : $(f \ x)$ devient "f x".
- ▶ Abstraction : $\lambda x. M$ devient "fun x -> M"

Notation :

- ▶ L'application est **associative à gauche**, donc

$$t_1 \ t_2 \ \dots \ t_n \text{ se lit } (\dots ((t_1 \ t_2) \ \dots) \ t_n).$$

- ▶ Une abstraction multiple

$$\lambda x_1 \ \dots \ x_n. t \text{ se lit } \lambda x_1. \ \dots \ \lambda x_n. t.$$

Définition inductive des termes du λ -calcul

$$\frac{x \text{ est une variable}}{x \text{ est un } \lambda\text{-terme}} \text{ (Var)}$$

$$\frac{t \text{ est un } \lambda\text{-terme} \quad u \text{ est un } \lambda\text{-terme}}{t \ u \text{ est un } \lambda\text{-terme}} \text{ (App)}$$

$$\frac{t \text{ est un } \lambda\text{-terme}}{\lambda x. t \text{ est un } \lambda\text{-terme}} \text{ (Abs)}$$

Variables libres et liées

Intuition : en rouge les variables libres et en bleu les variables liées :

$$\lambda x. (z \ x \ y \ (\lambda z. z \ y))$$

Les ensembles de variables libres et liées sont définis comme suit :

$$\begin{array}{llll} \text{fv}(x) & = & \{x\} & \text{bv}(x) & = & \emptyset \\ \text{fv}(t \ u) & = & \text{fv}(t) \cup \text{fv}(u) & \text{bv}(t \ u) & = & \text{bv}(t) \cup \text{bv}(u) \\ \text{fv}(\lambda x. t) & = & \text{fv}(t) \setminus \{x\} & \text{bv}(\lambda x. t) & = & \text{bv}(t) \cup \{x\} \end{array}$$

Ces ensembles ne sont pas disjoints, par exemple, pour $t = x \ (\lambda x. x)$, on a :

$$\text{fv}(x \ (\lambda x. x)) = \{x\} = \text{bv}(x \ (\lambda x. x))$$

Définition du remplacement par une variable fraîche

L'opération $t[x//y]$ désigne le **remplacement** de toutes les occurrences **libres** de x dans t par une variable **fraîche** y .

$$\begin{aligned}x[x//y] &= y \\z[x//y] &= z \\(t\ u)[x//y] &= t[x//y]\ u[x//y] \\(\lambda x.t)[x//y] &= \lambda x.t \\(\lambda z.t)[x//y] &= \lambda z.t[x//y]\end{aligned}$$

Conversion alpha

La relation $=_{\alpha}$, appelée **conversion alpha**, est la **congruence** engendrée par l'axiome $\sim_{\alpha}$. Formellement :

$$\frac{\lambda x.t \sim_{\alpha} \lambda y.t[x//y] \text{ où } y \text{ est une variable fraîche}}{\lambda x.t =_{\alpha} \lambda y.t[x//y]}$$
$$\frac{t =_{\alpha} t'}{\lambda x.t =_{\alpha} \lambda x.t'}$$
$$\frac{t =_{\alpha} t' \quad u =_{\alpha} u'}{t \ u =_{\alpha} t' \ u'}$$

Examples

$$\begin{aligned}\lambda x.(\lambda x.x z) &\sim_{\alpha} \lambda y.(\lambda x.x z) =_{\alpha} \lambda y.(\lambda y.y z) \\ &x (\lambda x.x) =_{\alpha} x (\lambda z.z)\end{aligned}$$

Convention de Barendregt sur les variables

À partir de maintenant, nous adoptons la convention suivante sur les variables :

1. Aucune variable n'est à la fois libre et liée.
2. Les variables liées portent toutes des noms distincts.

Exemple : $x (\lambda z.z)$ est **valide**, $x (\lambda x.x)$ ne l'est pas, $\lambda x.\lambda y.x z$ est **valide** mais $\lambda x.\lambda x.x z$ ne l'est pas.

Théorème (Convention de Barendregt) : Pour tout λ -terme t , il existe un λ -terme u satisfaisant la convention de Barendregt tel que $t =_{\alpha} u$.

En effet, $x (\lambda x.x) =_{\alpha} x (\lambda z.z)$ et $\lambda x.\lambda x.x z =_{\alpha} \lambda y.\lambda x.x z$.

Sémantique opérationnelle du λ -calcul

Une réduction β en une étape est définie inductivement par :

$$\begin{array}{c} \frac{}{(\lambda x.t) u \rightarrow_{\beta} t\{x/u\}} \qquad \frac{t \rightarrow_{\beta} t'}{\lambda x.t \rightarrow_{\beta} \lambda x.t'} \\[2ex] \frac{t \rightarrow_{\beta} t'}{t u \rightarrow_{\beta} t' u} \qquad \frac{u \rightarrow_{\beta} u'}{t u \rightarrow_{\beta} t u'} \end{array}$$

Mais que représente exactement l'opération $_ \{ _ / _ \}$?

Vers une définition correcte de la substitution

$$(\lambda x. (\lambda y. x)) y \rightarrow_{\beta} (\lambda y. x) \{x/y\} = \lambda y. y$$

Incorrect

$$(\lambda x. (\lambda y. x)) y =_{\alpha} (\lambda x. (\lambda z. x)) y \rightarrow_{\beta} (\lambda z. x) \{x/y\} = \lambda z. y$$

Correct

La notion de substitution d'ordre supérieur

La notation $t\{x/u\}$ signifie que l'on remplace **toutes** les occurrences **libres** de x dans t par u . Cette opération est définie à **α -conversion près** comme suit :

$$x\{x/u\} = u$$

$$y\{x/u\} = y$$

$$(\lambda y.v)\{x/u\} = \lambda y.v\{x/u\} \quad \text{si } x \neq y \text{ et } y \notin \text{fv}(u) \quad (\text{pas de capture de variable})$$

$$(t \ v)\{x/u\} = (t\{x/u\} \ v\{x/u\})$$

Exemple :

$$\blacktriangleright (\lambda y.(xy)x)\{x/3\} = \lambda y.(3y)3.$$

$$\blacktriangleright (\lambda y.x)\{x/y\} = \lambda z.y$$

Les séquences de réductions

Définition : Les **séquences de réduction** β sont obtenues par la **clôture réflexive et transitive** de la relation élémentaire $\rightarrow_\beta$. On la note $t \rightarrow_\beta^* t'$. Plus précisément :

$$t \rightarrow_\beta^* t' \quad \text{ssi} \quad t = t' \text{ ou } \exists t_1, \dots, t_n (t \rightarrow_\beta t_1 \rightarrow_\beta \dots \rightarrow_\beta t_n = t').$$

Example : Considérons

$$t_0 = (\lambda z.z) (\lambda y.y) w \rightarrow_\beta (\lambda y.y) w \rightarrow_\beta w = t_2$$

Alors $t_0 \rightarrow_\beta^* t_2$.

Une séquence de réductions β qui termine

$$\begin{aligned} & (\lambda x. \lambda f. x \ f \ y) \ (\lambda z. z) \ (\lambda w. w \ w) && \rightarrow_{\beta} \\ & (\lambda f. x \ f \ y) \ \{x / \lambda z. z\} \ (\lambda w. w \ w) && = \\ & (\lambda f. (\lambda z. z) \ f \ y) \ (\lambda w. w \ w) && \rightarrow_{\beta} \\ & (\lambda f. f \ y) \ (\lambda w. w \ w) && \rightarrow_{\beta} \\ & (f \ y) \ \{f / \lambda w. w \ w\} && = \\ & (\lambda w. w \ w) \ y && \rightarrow_{\beta} \\ & (w \ w) \ \{w / y\} && = \\ & y \ y \end{aligned}$$

Une séquence de réductions β qui ne termine pas

Soit $\Delta_f = \lambda x.f (x x)$.

$$\begin{aligned} \Delta_f \Delta_f &\rightarrow_{\beta} \\ (f (x x))\{x/\Delta_f\} &= \\ f (\Delta_f \Delta_f) &\rightarrow_{\beta} \\ f (f (x x))\{x/\Delta_f\} &= \\ f (f (\Delta_f \Delta_f)) &\rightarrow_{\beta} \\ \vdots & \end{aligned}$$

Normalisation Forte

Définition : Un terme t est **fortement normalisable**, noté $t \in SN_\beta$, s'il n'existe aucune séquence infinie de réductions β à partir de t , c'est-à-dire aucune séquence infinie de la forme suivante

$$t \rightarrow_\beta t_1 \rightarrow_\beta t_2 \rightarrow_\beta \cdots$$

Example : $(\lambda x. \lambda f. x \ f \ y) (\lambda z. z) (\lambda w. w \ w) \in SN_\beta$. et $\Delta_f \Delta_f \notin SN_\beta$.

Encodage des booléens

$\text{vrai} := \lambda x. \lambda y. x$ $\text{faux} := \lambda x. \lambda y. y$ $\text{ifte} := \lambda b. \lambda x. \lambda y. b \ x \ y$

Comportement :

$(\text{ifte vrai } M \ N) \rightarrow_{\beta}^* M$

$(\text{ifte faux } M \ N) \rightarrow_{\beta}^* N$

En OCaml:

```
let vrai = fun x -> fun y -> x;;  
# val vrai : 'a -> 'b -> 'a = <fun>  
let faux = fun x -> fun y -> y;;  
# val faux : 'a -> 'b -> 'b = <fun>  
let ifte = fun b -> fun x -> fun y -> b x y;;  
# val ifte : ('a -> 'b -> 'c) -> 'a -> 'b -> 'c = <fun>
```

Correspondance avec OCaml

```
let id = fun x -> x;;  
# val id : 'a -> 'a = <fun>  
let compose f g = fun x -> f (g x);;  
# val compose : ('a -> 'b) -> ('c -> 'a) -> 'c -> 'b = <fun>  
compose (fun x -> x + 1) (fun x -> 2 * x) 3;;  
# - : int = 7
```

Encodage des entiers (nombres de Church)

$$0 := \lambda f. \lambda x. x$$
$$1 := \lambda f. \lambda x. f \ x$$
$$2 := \lambda f. \lambda x. f(f \ x)$$
$$\text{succ} := \lambda n. \lambda f. \lambda x. f \ (n \ f \ x)$$

En OCaml :

```
let zero f x = x;;
```

```
let succ n = fun f -> fun x -> f (n f x);;
```

```
let one = succ zero;;
```

```
let plus n m = fun f -> fun x -> n f (m f x);;
```

Quelques propriétés du λ -calcul non typé

Théorème (Diminution des variables libres) : Si $t \rightarrow_{\beta} t'$, alors $fv(t) \supseteq fv(t')$.

Exemple: Soit $t = (\lambda x.x) y \rightarrow_{\beta} y = t'$. On a $fv(t) = \{y\} = fv(t')$.

Soit $t = (\lambda x.z) y \rightarrow_{\beta} z = t'$. On a $fv(t) = \{z, y\} \supset \{z\} = fv(t')$.

Théorème (Confluence) : La relation de réduction $\rightarrow_{\beta}$ est **confluente** : si $t \rightarrow_{\beta}^* t_1$ et $t \rightarrow_{\beta}^* t_2$, alors il existe t_3 tel que $t_1 \rightarrow_{\beta}^* t_3$ et $t_2 \rightarrow_{\beta}^* t_3$.

Exemple:

Soit $t = (\lambda x.xx) (id \ id)$, où $id := \lambda z.z$.

On a $t \rightarrow_{\beta} (id \ id) (id \ id) = t_1$ et $t \rightarrow_{\beta} (\lambda x.xx) id = t_2$.

Pour $t_3 = id \ id$, on ferme le diagramme avec $t_1 \rightarrow_{\beta}^* t_3$ et $t_2 \rightarrow_{\beta}^* t_3$.

Le lambda calcul typé

Isomorphisme de Curry-Howard

Système logique	$\Leftrightarrow$	Langage
Propositions	$\Leftrightarrow$	Types
Preuves	$\Leftrightarrow$	Programmes
Normalisation des preuves	$\Leftrightarrow$	Évaluation des programmes

Haskell Curry (1958), William Howard (1968)



Ajouter des (simples) types au λ -calcul

Grammaire des types :

$$A, B ::= \tau \quad \text{(types de base)} \quad | \\ A \rightarrow B \quad \text{(types fonctionnels)}$$

Notation : L'opérateur $\rightarrow$ est associatif à droite.

Ainsi, par exemple, $A_1 \rightarrow A_2 \rightarrow A_3$ signifie $A_1 \rightarrow (A_2 \rightarrow A_3)$.

Environnement de typage

- ▶ Un **environnement de typage** Γ est une **fonction finie** des variables vers les types, habituellement notée $x_1 : A_1, \dots, x_n : A_n$.
- ▶ Ainsi, par exemple, $x : A, y : B$ et $y : B, x : A$ sont deux notations différentes pour le même environnement de typage.
- ▶ Le **domaine** de $\Gamma = x_1 : A_1, \dots, x_n : A_n$, noté $\text{dom}(\Gamma)$, est l'ensemble $\{x_1, \dots, x_n\}$.
- ▶ On écrit $\Gamma, x : A$ pour l'environnement de typage qui étend Γ par la paire $x : A$. Cette opération n'est définie que si $x \notin \text{dom}(\Gamma)$.

Déduction naturelle comme λ -calcul **typé**

$$\frac{}{\Gamma, x : A \vdash x : A} \quad (ax)$$
$$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \rightarrow B} \quad (\rightarrow i) \qquad \frac{\Gamma \vdash t : A \rightarrow B \quad \Gamma \vdash u : A}{\Gamma \vdash t u : B} \quad (\rightarrow e)$$

On note $\Gamma \vdash_{\lambda} t : A$ la relation de typage (ou de dérivabilité). On dit que t est **typable** s'il existe Γ et A tels que $\Gamma \vdash_{\lambda} t : A$.

Remarque : $\Gamma \vdash t : A$ représente un *séquent* et $\Gamma \vdash_{\lambda} t : A$ représente une dérivation.

Exemples

- Un terme typable :

Nous avons $\vdash_{\lambda} (\lambda x.x)(\lambda y.y) : A$, pour $A := B \rightarrow B$. En effet, voici sa dérivation:

$$\frac{\frac{\frac{}{y : B \rightarrow B \vdash y : B \rightarrow B}}{\vdash \lambda y.y : (B \rightarrow B) \rightarrow (B \rightarrow B)}}{\frac{\frac{}{x : B \vdash x : B}}{\vdash \lambda x.x : B \rightarrow B}}{\vdash (\lambda y.y)(\lambda x.x) : B \rightarrow B}}$$

- Un terme non typable : $\lambda x.xx$.

Propriétés du système typé

Théorème (Réduction du sujet) : Si $\Gamma \vdash_{\lambda} t : A$ et $t \rightarrow_{\beta} t'$, alors $\Gamma \vdash_{\lambda} t' : A$.

Théorème (Normalisation Forte) : Tout terme **typable** est normalisant :
si $\Gamma \vdash_{\lambda} t : A$, alors $t \in SN_{\beta}$.

Conclusion

- ▶ Le λ -calcul fournit la base théorique des langages fonctionnels.
- ▶ Il permet de raisonner sur les programmes et leur exécution.
- ▶ Concepts clés : abstraction, application, substitution, réduction.
- ▶ OCaml est une extension pratique du λ -calcul pur.